

Guia de Go y del proyecto Arena Pokemon

Objetivo de esta guia

Esta guia esta pensada para una persona nueva que:

- No conoce Go.
- No conoce este repositorio.
- Necesita ser productiva rapido sin romper el proyecto.

Al terminar, deberias entender:

1. Como se ejecuta la app.
 2. Como esta organizada la arquitectura.
 3. Como fluye una request desde que entra hasta que responde.
 4. Como funciona la integracion con PokeAPI.
 5. Como esta armado el frontend embebido y el modo campeonato.
 6. Como testear y desplegar en Vercel.
-

1) Contexto rapido: que hace esta app

Arena Pokemon es una API HTTP en Go que:

- Expone endpoints de health y metadata.
- Consulta datos de Pokemon en PokeAPI.
- Sirve un frontend HTML/CSS/JS embebido en el backend.
- Muestra una galeria de Pokemon y simula un campeonato aleatorio en el cliente.

Endpoints principales:

- GET / -> frontend principal.
 - GET /health -> liveness.
 - GET /ready -> readiness.
 - GET /api/v1/info -> metadata de la app.
 - GET /api/v1/pokemon -> listado paginado.
 - GET /api/v1/pokemon/{name} -> detalle de un pokemon.
-

2) Mini introduccion a Go para este proyecto

No vamos a cubrir todo Go. Solo lo necesario para leer y mantener este codigo.

2.1 Estructura basica de archivo

Un archivo Go tipico:

```

package algo

import (
    "fmt"
)

func Saludar(nombre string) string {
    return fmt.Sprintf("Hola %s", nombre)
}

```

Ideas clave:

- `package`: modulo logico del archivo.
- `import`: dependencias.
- `func`: funciones.
- Si un nombre empieza en MAYUSCULA (`NewHandler`), es exportado.
- Si empieza en minuscula (`writeJSON`), es interno al package.

2.2 Structs y metodos

Una struct agrupa estado:

```

type Handler struct {
    cfg *config.Config
}

```

Metodo con receiver:

```

func (h *Handler) RegisterRoutes(mux *http.ServeMux) {
    // usa h.cfg
}

```

2.3 Interfaces implicitas

Go no usa `implements` explicito. Si un tipo tiene los metodos requeridos, ya cumple la interfaz.

Ejemplo del proyecto:

- `http.Handler` requiere `ServeHTTP(w, r)`.
- `http.HandlerFunc` permite usar funciones como handlers.

2.4 Manejo de errores

Patron comun:

```

res, err := algo()
if err != nil {
    return err
}

```

En este repo se usa mucho `fmt.Errorf("...: %w", err)` para envolver errores.

2.5 Concurrency basica

En `cmd/api/main.go` se lanza el servidor en goroutine:

```
go func() {  
    serverErr <- srv.Start()  
}()
```

Esto permite esperar simultaneamente:

- Error del servidor.
 - Senal del sistema para shutdown.
-

3) Estructura del repositorio (mapa mental)

```
.  
  api/                                # Entrada para Vercel Functions (serverless)  
  cmd/api/                            # Entrada del servidor HTTP tradicional  
  internal/  
    config/                          # Carga de variables de entorno  
    handler/                         # Endpoints HTTP + frontend embebido  
    middleware/                      # RequestID, logging, recover panic  
    pokeapi/                         # Cliente HTTP hacia pokeapi.co  
    server/                          # Construcccion del http.Server y cadena handler  
  pkg/logger/                        # Inicializacion de slog  
  README.md  
  vercel.json
```

Regla de arquitectura:

- `cmd/` = entypoints.
 - `internal/` = logica de negocio y web interna del modulo.
 - `pkg/` = utilidades reutilizables fuera de `internal`.
-

4) Flujo completo de una request

Ejemplo con GET `/api/v1/pokemon/pikachu`:

1. Entra por servidor en `cmd/api/main.go` (local) o funcion `api/index.go` (Vercel serverless mode si aplica).
2. Se construye handler global con `internal/server/NewHandler(...)`.
3. Pasan middlewares:
 - `RequestID`
 - `Logger`

- Recover
4. `RegisterRoutes` en `internal/handler/handler.go` deriva al handler correcto.
 5. `GetPokemon` en `internal/handler/pokemon.go` valida parametro.
 6. Llama al cliente `pokeClient.GetPokemon(...)`.
 7. `internal/pokeapi/client.go` hace request a PokeAPI.
 8. Se mapean modelos y se responde JSON con `writeJSON`.
-

5) Capa HTTP: handlers

5.1 Handler (dependencias)

`internal/handler/handler.go` define:

- Configuración (`cfg`).
- Logger (`log`).
- Cliente externo (`pokeClient`).

Esto evita usar variables globales y facilita testear.

5.2 Registro de rutas

`RegisterRoutes` centraliza el mapa de endpoints.

Ventaja: sabes rapidamente que expone la API sin buscar en muchos archivos.

5.3 Utilidades de respuesta

- `writeJSON` fija content type y serializa.
 - `writeError` devuelve payload uniforme: `{"error": "..."}.`
-

6) Cliente PokeAPI

`internal/pokeapi/client.go` contiene:

- Construcción de requests HTTP.
- Timeouts.
- Manejo de status (200, 404, etc).
- Parseo de JSON externo a modelo interno.

Puntos importantes:

- Usa `http.NewRequestWithContext` para cancelar con context.
 - Evita `http.DefaultClient`; crea client propio con timeout.
 - Devuelve `ErrNotFound` para 404 y permite manejo especifico en handlers.
-

7) Frontend embebido y campeonato

El frontend vive en `internal/handler/frontend.go` como string HTML grande.

7.1 Que hace el endpoint /

- Setea `Content-Type: text/html`.
- Renderiza `frontendHTML`.
- Inserta nombre de app (`__APP_NAME__`).

7.2 En cliente (JavaScript)

- Carga lotes de pokemon desde `/api/v1/pokemon`.
- Renderiza cards con imagen y nombre.
- Permite filtrar por texto.
- Inicia campeonato aleatorio:
 - Seleccion de participantes.
 - Batallas con energia/momentum.
 - Ganador por ronda.
 - Historial y campeon final.

7.3 Decisiones de diseno

- Se evita framework frontend para mantener despliegue simple.
 - Todo se sirve desde el backend Go.
 - UX dinamica sin build step de JS.
-

8) Configuracion por entorno

Archivo local `.env` (desarrollo):

- `APP_NAME`, `APP_ENV`, `APP_VERSION`
- `SERVER_HOST`, `SERVER_PORT`
- timeouts de servidor
- `LOG_LEVEL`

En Vercel:

- Debes usar `APP_ENV=production`.
 - Puerto lo da Vercel via `PORT`.
 - El codigo ya prioriza `PORT` y usa `SERVER_PORT` solo como fallback local.
-

9) Logging y observabilidad

Se usa `log/slog`:

- En `production`: JSON (`NewJSONHandler`).
- En `development`: texto legible (`NewTextHandler`).

Middlewares agregan informacion util:

- `metodo`
- `path`
- `status`
- `duracion`
- `request_id`

Esto ayuda a depurar errores de forma rapida.

10) Manejo de errores y robustez

Buenas practicas aplicadas:

- Validacion de input (`pokemonNameRE`, parse de query params).
- Traduccion de errores externos a HTTP status apropiados.
- `Recover` middleware para panics.
- Timeouts de lectura/escritura/idle en servidor.

11) Testing en este proyecto

Tipos de tests presentes:

- Unit tests de handlers HTTP.
- Tests del cliente PokeAPI con `httptest.Server` (mock upstream).

Comandos utiles:

```
make test
make test-cover
go test ./...
```

Consejo para nuevos tests:

1. Usa `httptest.NewRecorder` + `httptest.NewRequest` para handlers.
2. Simula PokeAPI con `httptest.NewServer`.
3. Verifica status code, headers y body.

12) Ejecutar local

```
cp .env.example .env
make tidy
make run
```

Abrir:

- `http://localhost:8080/` para frontend.
 - `http://localhost:8080/health` para health.
-

13) Deploy en Vercel

Este repo ya incorpora `vercel.json` para framework Go.

Flujo recomendado:

1. Push a rama principal.
2. Importar proyecto en Vercel.
3. Verificar framework `go`.
4. Definir variables (al menos `APP_ENV=production`).
5. Deploy.

Si aparece error de bind en puerto:

- Revisar que app este usando `PORT` (ya implementado).
 - Revisar logs de deploy para confirmar endpoint ejecutado.
-

14) Checklist de onboarding (primeros 7 dias)

Dia 1

- Levantar proyecto local.
- Probar endpoints base.
- Leer `README.md` y esta guía.

Dia 2

- Recorrer handlers y middlewares.
- Entender flujo request/response.

Dia 3

- Revisar cliente `pokeapi`.
- Agregar log extra de prueba (sin commitear) para entender trazas.

Dia 4

- Escribir un test nuevo simple para un caso de error de handler.

Dia 5

- Modificar UI del frontend embebido (texto, estilos o tarjeta).

Dia 6

- Hacer un mini cambio full-stack (handler + frontend + test).

Dia 7

- Ejecutar deploy de prueba en Vercel.
 - Documentar hallazgos en PR.
-

15) Glosario rapido

- **Handler:** funcion/metodo que responde una ruta HTTP.
 - **Middleware:** capa intermedia que envuelve handlers.
 - **Context:** objeto para cancelacion, timeouts y metadatos por request.
 - **ServeMux:** router HTTP estandar de Go.
 - **Struct:** tipo de datos compuesto en Go.
 - **Pointer (*):** referencia a un valor para evitar copias y permitir mutaciones.
-

16) Recomendaciones para contribuir sin romper

1. Cambios pequenos y atomicos.
 2. Testear siempre con `go test ./...` antes de push.
 3. Mantener respuestas de error consistentes.
 4. Evitar side-effects globales.
 5. Documentar decisiones de arquitectura en PR.
-

17) Siguiente nivel (plan de aprendizaje)

Cuando ya domines esta base:

- Aprender interfaces y composition en Go mas a fondo.
 - Practicar profiling (`pprof`) para rendimiento.
 - Agregar caché para consultas repetidas a PokeAPI.
 - Separar frontend embebido en assets versionados si crece mucho.
-

Resumen final

Este proyecto es una excelente base para aprender Go aplicado a backend real:

- HTTP server limpio con `stdlib`.
- Integracion externa con cliente tipado.

- Middlewares utiles para produccion.
- Frontend integrado para experiencia completa.
- Pipeline de tests y despliegue.

Si eres nuevo en Go, no intentes aprender todo de una vez. Sigue el flujo de una request end-to-end y haz cambios pequenos con tests. Esa es la forma mas rapida y segura de ganar velocidad en este codigo.